

TheMakerLab - Informe (Entrega 1)

Participantes:

- Juan Felipe Ortiz Cardenas
- Thomas Holguín González
- Markus McMulloch
- Diego Oxman

Alcance

Este informe muestra el modelado orientado a objetos para la primera entrega del proyecto integrador MakerLab EIA, sistema de consola para la administración de usuarios, laboratorios, equipos, certificaciones, reservas, sesiones de uso y mantenimiento. El desarrollo se limita al diseño conceptual y estructural del sistema: el diagrama de clases UML, la definición de jerarquías de herencia entre las entidades del dominio (por ejemplo, tipos de usuario o de equipo) y la aplicación de polimorfismo en los comportamientos que varían según esas jerarquías. La representación de los datos se implementa mediante arreglos de capacidad fija (n), definida previamente para cada colección de objetos del sistema.

Requisitos funcionales

RF1: Registrar usuarios

Clase abstracta con tres tipos de comportamiento

ServicioAdministrativo.registrarUsuario

RF2: Registrar laboratorios

Clase laboratorio con su constructor y 5 datos

ServicioAdministrativo.agregarLaboratorio

RF3: Registrar equipos

Clase abstracta con tres subtipos y con constructor

ServicioAdministrativo.registrarEquipoEnLaboratorio

RF4: Registrar certificaciones

Clase certificación con constructor

ServicioAdministrativo.registrarCertificaccion

RF5: Crear reservas

Clase reserva con constructor

ServicioAdministrativo.registrarReserva

RF6: Validar que no halla cruces

Cadena de reglas (5 reglas)

ValidadorCertificacionUsuario.validar

ValidadorCertificacionUsuarioBasico.validar

Usuario.tieneCertificacionVigente

validadorReservaEquipoLaboratorio.verificar

ValidadorDisponibilidadEquipoLaboratorio.verificar

EquipoLaboratorio.estaOperativo

FechaSistema.hayCruceHorario

Reserva.seCruzaCon

RF7: Lista de espera para solicitudes válidas

Arreglo **listaEspera[]**

ServicioAdministrativo.moverAListaDeEspera

RF8: Abrir y Cerrar sesión para una reserva valida

Clase SesionUso con constructor

ServicioAdministrativo.abrirSesion

RF9: Tickets

Clase TicketMantenimiento con constructor y prioridad

ServicioAdministrativo.crearTicket

RF10: Bloquear nuevas reservas al haber una falla de un equipo.

Al **crearTicket(...)** pone el equipo **EN_MANTENIMIENTO**

bloquearReservasFuturas(e, m)

Requisitos no funcionales

RNF1: Cerrar ticket solo con acción registrada

Regla en Modelo

TicketMantenimiento.cerrar(estadoFinal)

RNF2: Consultas

consultarDisponibilidad(idLab, fecha, horaInicio, horaFin)

consultarReservas(idUsuario)

consultarEquiposFueraDeServicio()

consultarCertificacionesPorVencer(días)

Certificacion.diasParaVencer()

consultarTicketsPorPrioridad(prioridad)

RNF3: Indicadores en memoria

ServicioAdministrativo.mostrarIndicadores()

RNF4: Demo

MakerLabApp.main(...)

Reglas de negocio

RN1: Identificadores únicos

ServicioAdministrativo · registrar*/buscar*

RN2: Hora final > inicial y dentro del horario

registrarReserva + FechaSistema.dentroDelHorario

RN3: Certificación vigente y del tipo

ValidadorCertificacionUsuarioBasico.validar → Usuario/Certificación

RN4: Equipo no operativo → sin reservas

EquipoLaboratorio.estaOperativo

RN5: Ticket cerrado conserva acción y técnico

TicketMantenimiento.cerrar

RN6: Lista de espera sin duplicados

moverAListaDeEspera

RN7: Sesión desde confirmada; sin doble cierre

abrirSesion + SesionUso.cerrar

RN8: Siguiendo candidato por prioridad

Lista FIFO (parcial) + interfaces

Riesgos del código

R1: N fija y compartida; rechazadas ocupan sitio

ServicioAdministrativo.java

R2: Historial se sobrescribe al llenarse

TicketMantenimiento.java

R3: Fechas imposibles

FechaSistema.java

R4: Getters exponen referencias

EquipoLaboratorio.java

ServicioAdministrativo.java

R5: System.out

ConsolaUtil.java

Riesgos que tomamos como estudiantes en el modelado

Herencia con clases abstractas y polimorfismo. Usuario y EquipoLaboratorio son abstractas y sus subtipos concretos redefinen el comportamiento propio; un discriminador String de tipo permite además clasificar sin instanceof.

ServicioAdministrativo como fachada única. Una sola clase concentra todas las colecciones y todas las reglas que cruzan varias entidades, aplicadas en un orden fijo, y sirve de punto de entrada único.

Validaciones de reserva en interfaces inyectables. La comprobación de certificación y la de disponibilidad viven detrás de interfaces cuyas implementaciones se inyectan por constructor, de modo que el servicio depende de abstracciones y no de clases concretas (bajo acoplamiento).

Multiplicidades 0..* como arreglo de capacidad n + contador. Cada clase contenedora recibe n por constructor —no como constante interna— para que la capacidad ingresada por teclado sea visible y trazable; la identidad se basa en un idpropio y las búsquedas son lineales.

Muchas Gracias